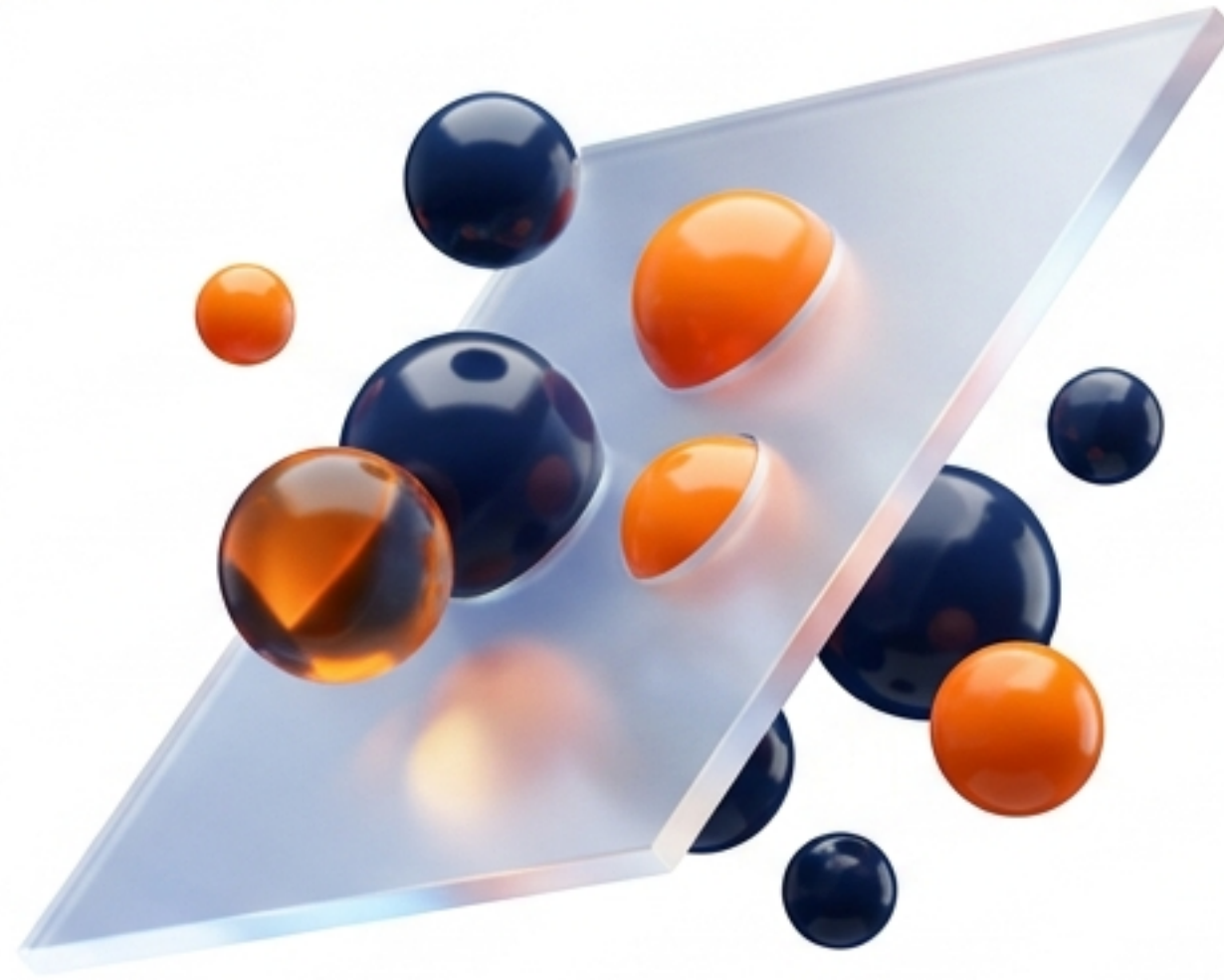


# Chương 5: Máy học Véc-tơ Hỗ trợ (Support Vector Machines)



Giảng viên  
Đại học Đông Á  
August 11, 2026

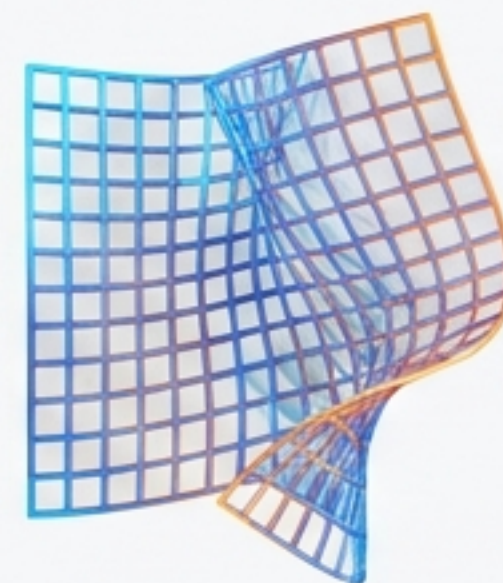
# Mục tiêu của Chương



Hiểu khái niệm cốt lõi của Support Vector Machines (SVM).



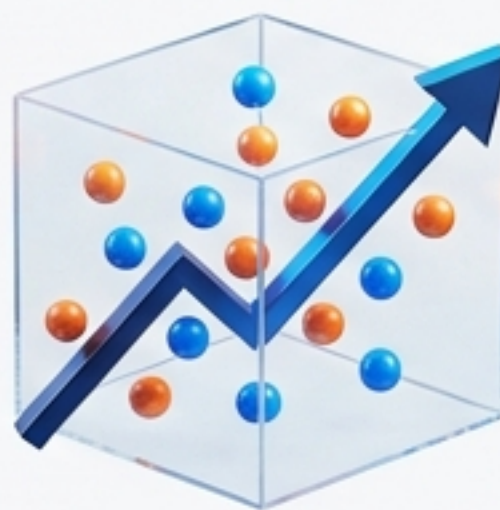
Phân biệt giữa Biên cứng (Hard Margin) và Biên mềm (Soft Margin) trong phân loại SVM tuyến tính.



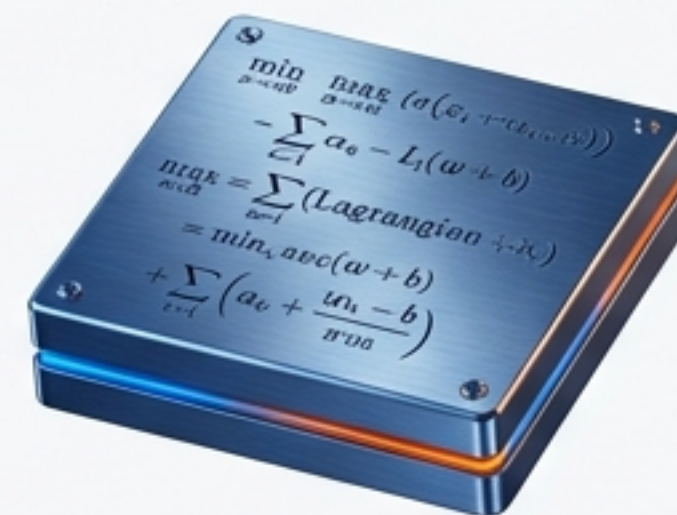
Xử lý dữ liệu phi tuyến bằng kỹ thuật bổ sung đặc trưng hoặc Kernel trick.



Áp dụng Kernel Đa thức (Polynomial) và Kernel Gaussian RBF.



Khám phá cách sử dụng SVM cho bài toán Hồi quy (Regression).



Đi sâu vào toán học phía sau SVM: Bài toán tối ưu, Hàm mục tiêu và Đối ngẫu.

# Mục lục



1. Phân loại SVM Tuyến tính



2. Phân loại SVM Phi tuyến



3. Lớp SVM & Hồi quy SVM



4. Toán học: Bên trong Thuật toán SVM

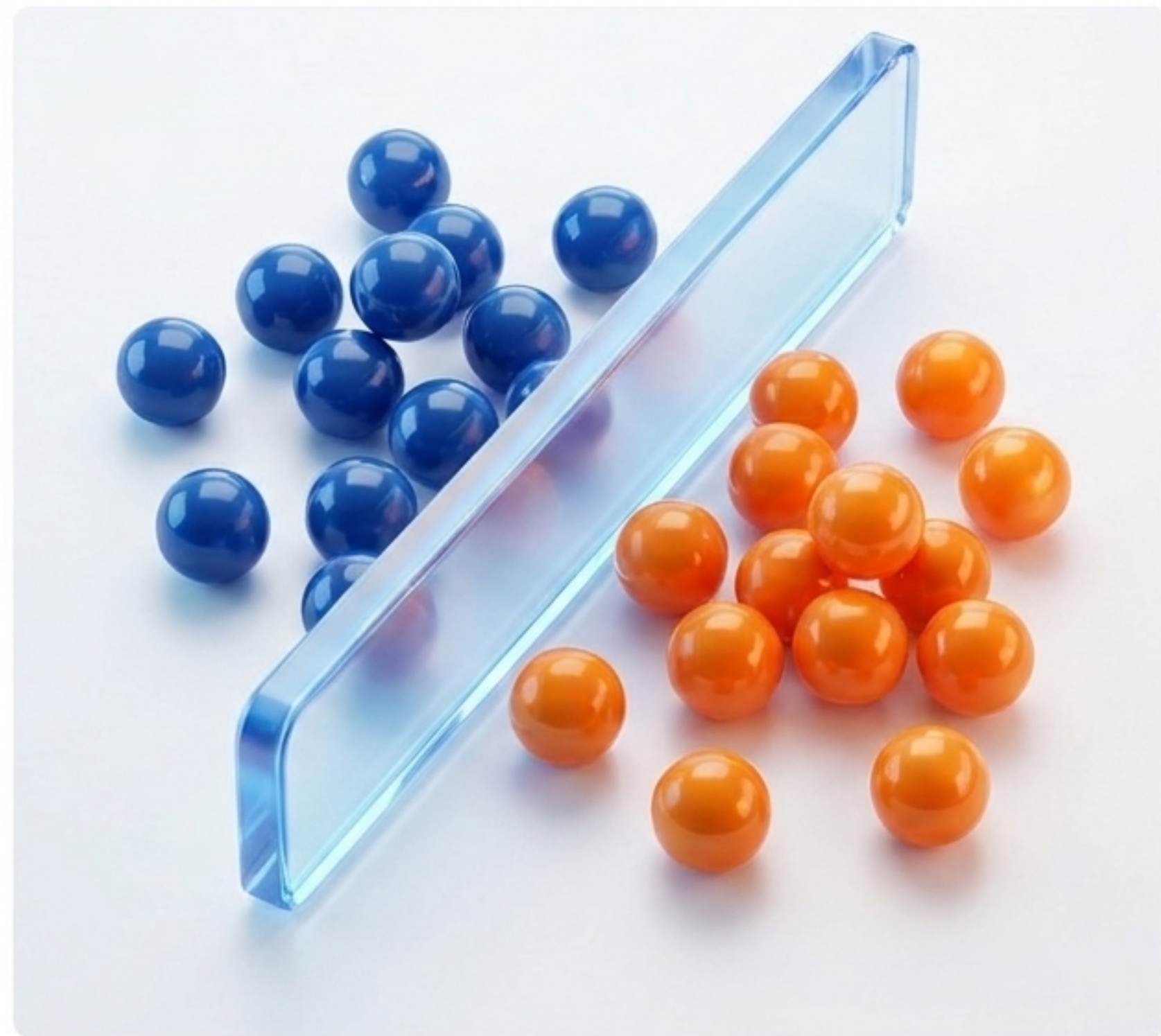


5. Tổng kết chương

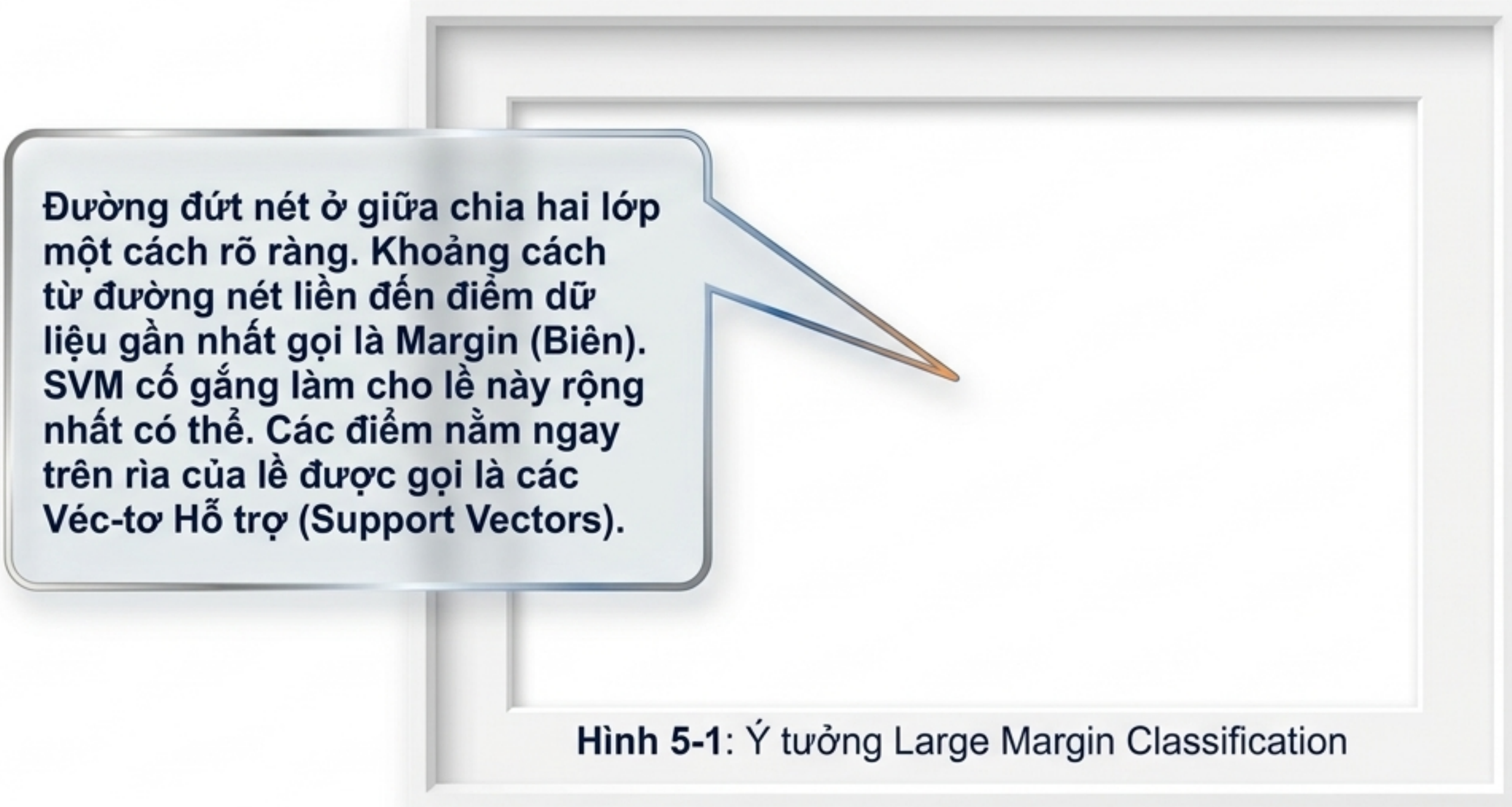
# 1.1 Ý tưởng cơ bản của SVM

SVM là một trong những mô hình phổ biến và mạnh mẽ nhất trong Học máy truyền thống. SVM có thể giải quyết các bài toán phân loại tuyến tính, phi tuyến, và cả hồi quy. Tốt nhất cho các tập dữ liệu có kích thước vừa và nhỏ nhưng cực kỳ phức tạp.

Ý tưởng chính: Tìm ra một đường thẳng (hoặc siêu phẳng) chia tách các lớp dữ liệu sao cho khoảng cách từ đường thẳng tới các điểm gần nhất của mỗi lớp là lớn nhất.



## 1.2 Mô hình Phân loại Biên Lớn (Large Margin Classification)



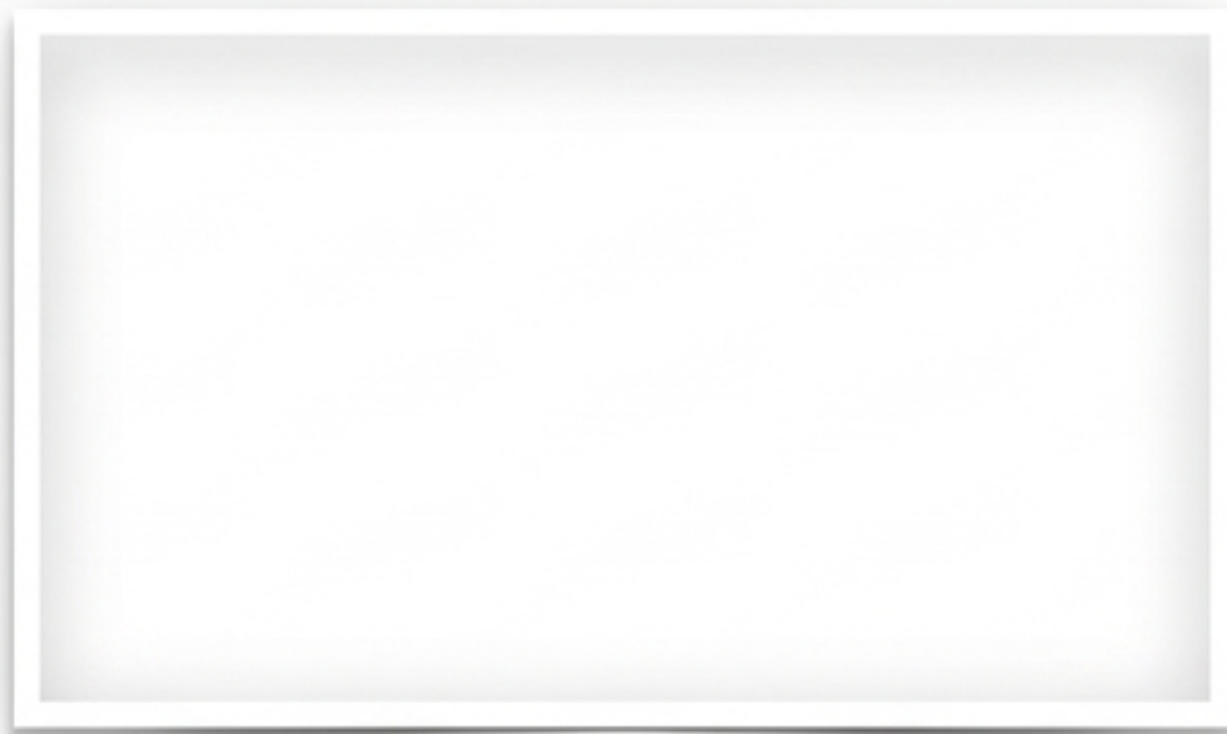
Đường đứt nét ở giữa chia hai lớp một cách rõ ràng. Khoảng cách từ đường nét liền đến điểm dữ liệu gần nhất gọi là Margin (Biên). SVM cố gắng làm cho lề này rộng nhất có thể. Các điểm nằm ngay trên rìa của lề được gọi là các Véc-tơ Hỗ trợ (Support Vectors).

Hình 5-1: Ý tưởng Large Margin Classification

## 1.3 Tính nhạy cảm với Tỷ lệ Đặc trưng (Feature Scaling)



SVM rất nhạy cảm với tỷ lệ của các đặc trưng (thang đo). Nếu các đặc trưng có **tỷ lệ quá chênh lệch** (VD:  $x_1$  từ 0-1000,  $x_2$  từ 0-1), ranh giới quyết định sẽ gần như song song với trục lớn hơn. Do đó, luôn cần thực hiện **StandardScaler** trước khi đưa dữ liệu vào SVM.



Hình 5-2: Sự khác biệt của lề khi chưa scale và đã scale

## 1.4 Phân loại Biên Cứng (Hard Margin Classification)



Cố gắng chia tách tất cả các điểm một cách hoàn hảo tuyệt đối. Không điểm nào được phép nằm trong vùng lề hoặc nằm sai phía ranh giới.

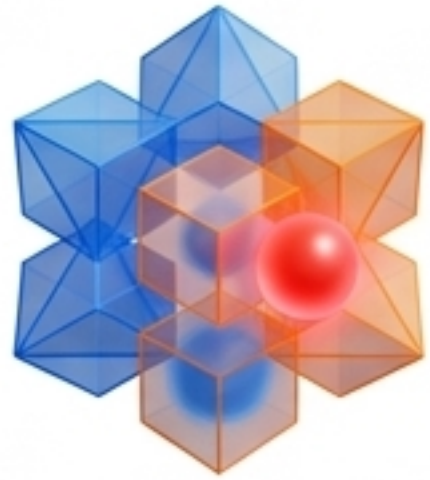
### Vấn đề 1:

Chỉ hoạt động được khi dữ liệu phân tách tuyến tính (Linearly Separable).

### Vấn đề 2:

Rất nhạy cảm với Điểm bất thường (Outliers). Một nhiễu nhỏ cũng có thể làm biến dạng ranh giới.

## 1.5 Tác động của Điểm bất thường (Outliers)



Hình bên trái: Một **điểm xanh** rơi vào vùng **lớp đỏ** khiến thuật toán không thể vẽ được đường thẳng chia cách hoàn hảo.

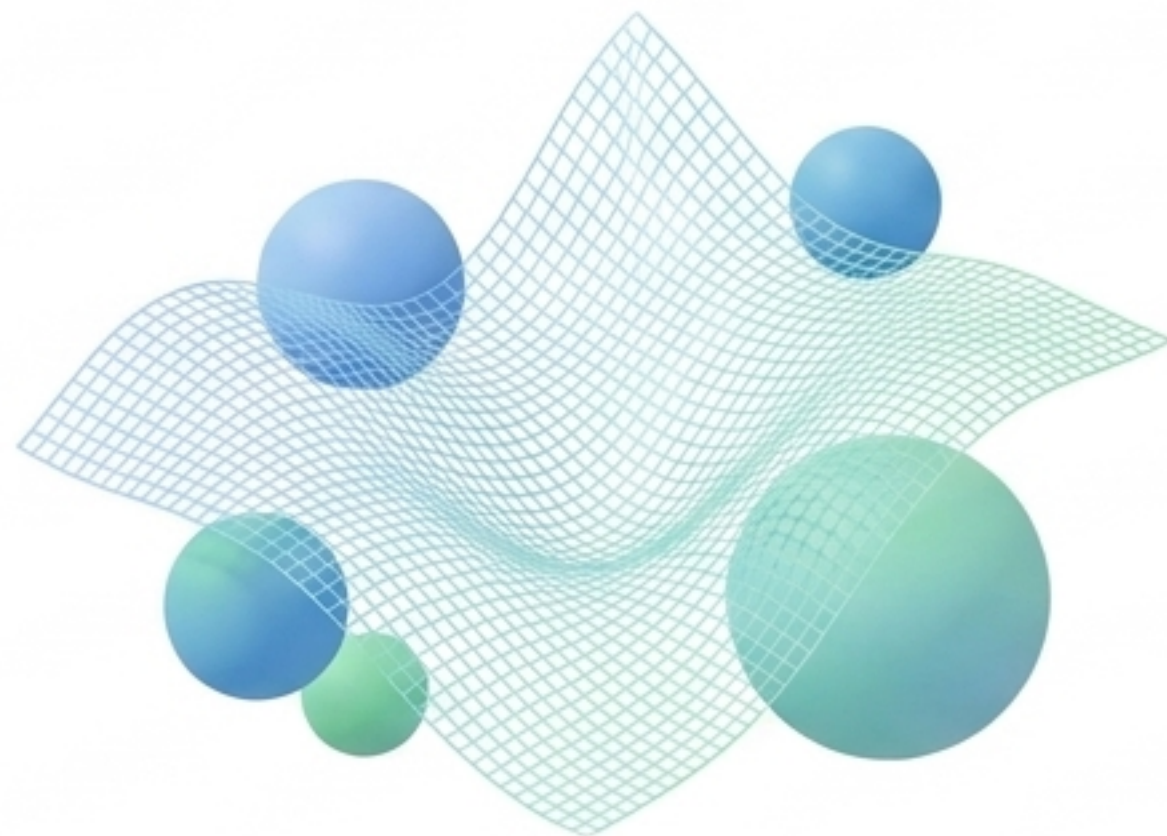


Hình 5-3: **Biên Cứng** bị bóp méo hoặc thất bại khi có outlier



Hình bên phải: Một **outlier** làm lề bị co hẹp lại một cách **cực đoan**, mô hình **mất tính tổng quát**.

## 1.6 Phân loại Biên Mềm (Soft Margin Classification)



Để tránh những vấn đề của Biên cứng, ta sử dụng một mô hình linh hoạt hơn.  
Mục tiêu mới: Tìm sự cân bằng giữa việc giữ cho lề rộng nhất có thể và hạn chế tối đa các điểm vi phạm lề (Margin Violations).  
Đây gọi là **Phân loại Biên mềm (Soft Margin Classification)**.

# 1.7 Siêu tham số C trong Scikit-Learn

Tham số '**C**' điều khiển sự cân bằng trong Biên mềm.

## **C nhỏ:**

Lề rộng hơn, nhưng cho phép nhiều điểm vi phạm lề (Tăng bias, Giảm variance). Dễ bị Underfitting.

## **C lớn:**

Lề hẹp lại để hạn chế vi phạm lề (Giảm bias, Tăng variance). Dễ bị Overfitting.



Hình 5-4: So sánh khi C thấp (trái) và C cao (phải)

## 1.8 Triển khai LinearSVM bằng Python



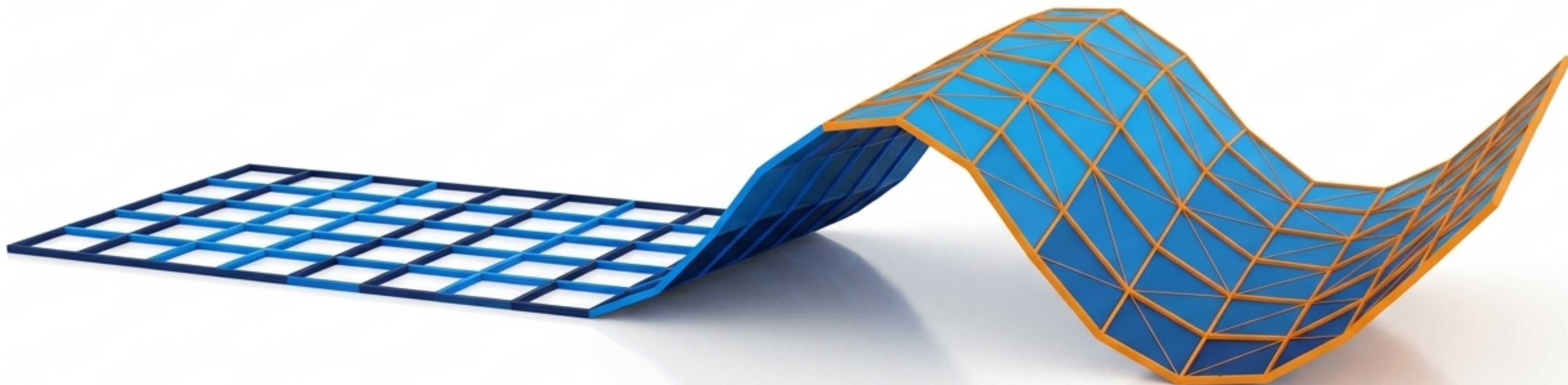
```
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

# Tải dữ liệu Iris (Hoa diem vi)
iris = datasets.load_iris()
X = iris["data"][:, (2, 3)] # Chiều dài, chiều rộng cánh hoa
y = (iris["target"] == 2).astype(np.float64) # Iris virginica

# Xây dựng Pipeline: chuẩn hóa + huấn luyện
svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("linear_svc", LinearSVC(C=1, loss="hinge", random_state=42))
])

svm_clf.fit(X, y)
print("Dự đoán:", svm_clf.predict([[5.5, 1.7]])) # Output: [1.]
```

## 2.1 Giới thiệu SVM Phi tuyến



Rất nhiều bộ dữ liệu thực tế **không thể phân tách bằng một đường thẳng**. Một rất nhiều bộ dữ liệu thực tế không thể phân tách bằng một đường thẳng. Một phương pháp đơn giản để xử lý tập dữ liệu phi tuyến là **thêm nhiều đặc trưng (Features)** mới, chẳng hạn như đặc trưng **đa thức (Polynomial)**. Dữ liệu gốc ở 1D có thể không tách được, nhưng khi chiếu lên không gian 2D, ta có thể dùng đường thẳng để tách!

## 2.2 Minh họa Thêm đặc trưng đa thức

**Hình bên trái:** Các điểm đỏ và xanh trên một trục 1D ( $x_1$ ). Rõ ràng **không** có 1 điểm cắt nào **phân chia hoàn hảo**.



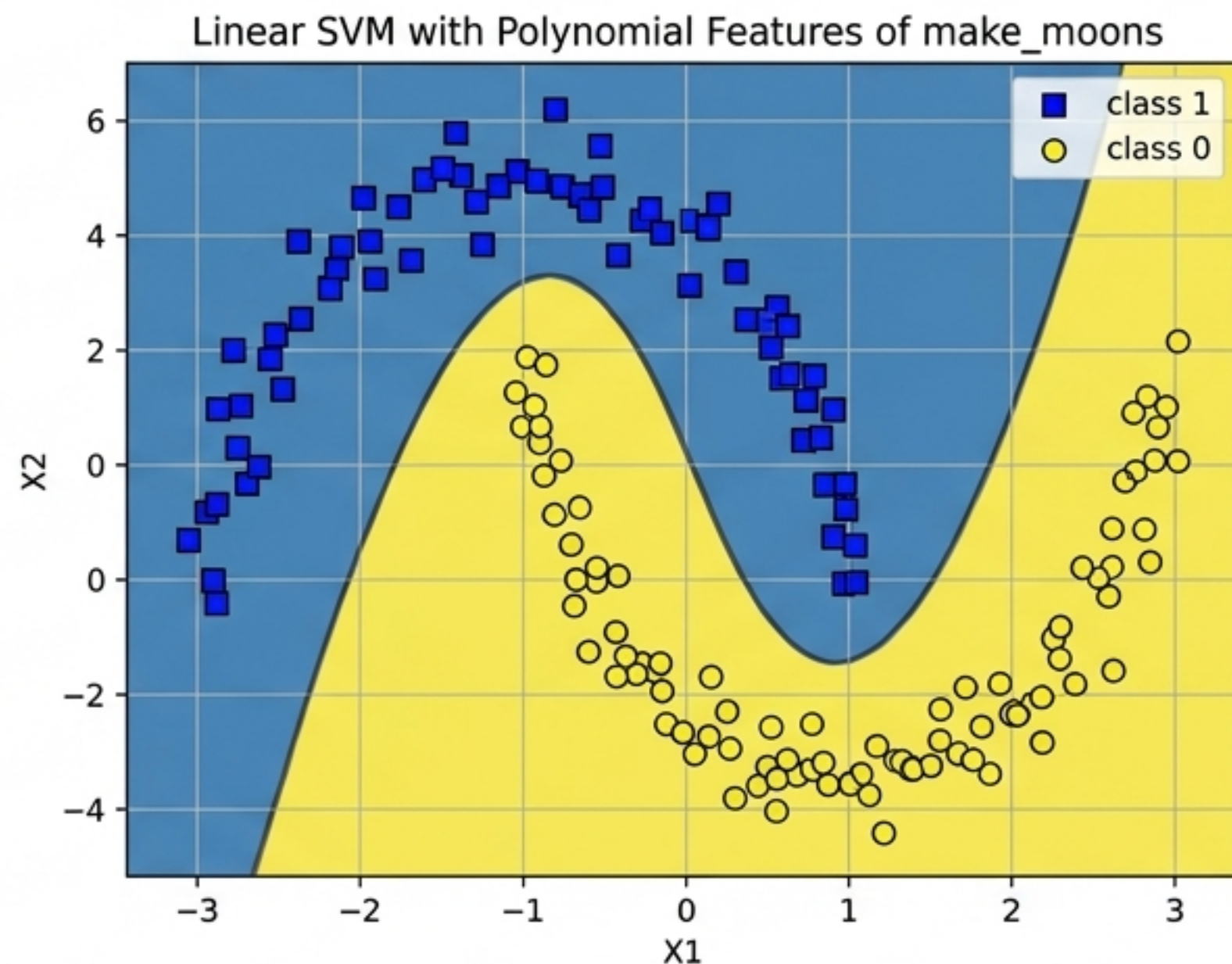
Khi ta thêm đặc trưng mới  $x_2 = (x_1)^2$ .

**Hình bên phải:** Trong không gian 2D ( $x_1, x_2$ ), dữ liệu đã trở nên **phân tách tuyến tính**.

Hình 5-5: Thêm đặc trưng  $x_2 = x_1^2$  giúp phân tách tuyến tính

## 2.3 Phân tách Dataset "Moons" (Hai nửa vành trăng)

Bộ dữ liệu Make Moons gồm 2 nửa vành khuyên lồng vào nhau. Sử dụng **'PolynomialFeatures'** kết hợp **'StandardScaler'** và **'LinearSVC'**. Đường cong ranh giới quyết định rất linh hoạt và phân loại chính xác các điểm.



Hình 5-6: Ranh giới quyết định của Linear SVM với Polynomial Features

## 2.4 Kernel Trick (Thủ thuật Kernel)



Việc tạo ra đặc trưng đa thức có bậc cao (Degree = 10, 20) sẽ tạo ra vô số các đặc trưng mới, làm chậm mô hình (**Khủng hoảng chiều không gian - Curse of Dimensionality**). May mắn thay, với SVM ta có một **phép màu toán học** gọi là **Kernel trick**. **Kernel Trick** cho phép SVM tính toán ra kết quả phân loại giống y hệt như việc ta thêm hàng tỉ đặc trưng đa thức, nhưng thực tế nó không hề thêm một đặc trưng nào. Do đó, **không làm chậm thuật toán!**